Project T20 Cricket Data Visualization Using Python, Power BI and Tableau

AIM

To analyze the T20 Cricket dataset and create three visualizations - bar chart, line chart, and pie chart - using Python, Power BI, and Tableau to understand batting performance, identify top run scorers, and analyze match results across teams.

PROBLEM STATEMENT

T20 cricket generates large amounts of data such as batsman runs, balls faced, match dates, and match winners. Understanding this data manually is difficult. This project uses Python libraries such as Pandas and Matplotlib, along with Power BI and Tableau, to process the T20 Cricket dataset and create visualizations. These visualizations help identify top run scorers, observe run trends across match dates, and understand how match wins are distributed among teams.

SOFTWARE REQUIREMENTS

- Platform: Google Colab, Power BI Desktop, Tableau Desktop
- Programming Language: Python
- Dataset Format: CSV (.csv)
- Libraries: Pandas, Matplotlib
- Input Dataset: T20 Cricket CSV files (df_batting.csv and df_match.csv)

DATASET DESCRIPTION

The T20 Cricket dataset contains 699 batting records from 45 matches played between 16 Oct 2022 and 13 Nov 2022. The batting file (df_batting.csv) and the match file (df_match.csv) are linked using match_id. The main columns used are given below.

Column Name	Description
match_id	Unique ID of the match (e.g., T20I # 1823) - common to both files
batsmanName	Name of the batsman (df_batting.csv)
runs, balls	Runs scored and balls faced by the batsman
4s, 6s, SR	Number of fours, sixes, and strike rate
teamInnings	Team batting in the innings
matchDate	Date on which the match was played (df_match.csv)
winner	Team that won the match (or no result / abandoned)
team1, team2, ground, margin	Teams, venue and winning margin of the match

1. PYTHON

PROCEDURE

1. Import the dataset into Python, Power BI, and Tableau.

2. Inspect the dataset and identify the match date, batsman name, runs, and winner columns.
3. Handle missing values and ensure that dates and numerical columns have the correct data types.
4. Merge the batting and match files using match_id, and calculate total runs for each batsman.
5. Create a bar chart to compare values across categories.
6. Create a line chart to analyze trends.
7. Create a pie chart to show the distribution across categories.
8. Use Pandas and Matplotlib to generate the three visualizations in Python.
9. Import the dataset into Power BI and Tableau and recreate the same visualizations.
10. Compare the charts and identify trends, popular categories, and business insights.

Program and Output

```
import pandas as pd
import matplotlib.pyplot as plt

# Show all columns when printing
pd.set_option("display.max_columns", None)
pd.set_option("display.width", 100)

# Read the CSV files
batting = pd.read_csv("df_batting.csv")
match = pd.read_csv("df_match.csv")

# Display first five rows
print(batting.head())

# Display column names
print(batting.columns.tolist())
print(match.columns.tolist())

# Check dataset information
print(batting.info())
print(match.info())
```

Output:

```
                  match teamInnings  battingPos             batsmanName  runs  balls  4s  6s  \
0  Namibia Vs Sri Lanka     Namibia           1      Michael van Lingen     3      6   0   0
1  Namibia Vs Sri Lanka     Namibia           2           Divan la Cock     9      9   1   0
2  Namibia Vs Sri Lanka     Namibia           3  Jan Nicol Loftie-Eaton    20     12   1   2
3  Namibia Vs Sri Lanka     Namibia           4           Stephan Baard    26     24   2   0
4  Namibia Vs Sri Lanka     Namibia           5      Gerhard Erasmus(c)    20     24   0   0

       SR out/not_out     match_id
0   50.00         out  T20I # 1823
1  100.00         out  T20I # 1823
2  166.66         out  T20I # 1823
3  108.33         out  T20I # 1823
4   83.33         out  T20I # 1823
['match', 'teamInnings', 'battingPos', 'batsmanName', 'runs', 'balls', '4s', '6s', 'SR', 'out/not_out', 'match_id']
['team1', 'team2', 'winner', 'margin', 'ground', 'matchDate', 'match_id']
<class 'pandas.DataFrame'>
RangeIndex: 699 entries, 0 to 698
Data columns (total 11 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   match        699 non-null    str
 1   teamInnings  699 non-null    str
 2   battingPos   699 non-null    int64
 3   batsmanName  699 non-null    str
 4   runs         699 non-null    int64
 5   balls        699 non-null    int64
 6   4s           699 non-null    int64
 7   6s           699 non-null    int64
 8   SR           699 non-null    str
 9   out/not_out  699 non-null    str
 10  match_id     699 non-null    str
dtypes: int64(5), str(6)
memory usage: 60.2 KB
None
<class 'pandas.DataFrame'>
RangeIndex: 45 entries, 0 to 44
```

```
Data columns (total 7 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   team1      45 non-null     str
 1   team2      45 non-null     str
 2   winner     45 non-null     str
 3   margin     41 non-null     str
 4   ground     45 non-null     str
 5   matchDate  45 non-null     str
 6   match_id   45 non-null     str
dtypes: str(7)
memory usage: 2.6 KB
None
```

```
 # Clean column names
 batting.columns = batting.columns.str.strip()
 match.columns = match.columns.str.strip()

 # Clean batsman names (remove captain tag and wrong symbols)
 batting["batsmanName"] = (
     batting["batsmanName"]
     .str.replace("(c)", "", regex=False)
     .str.replace("â€", "", regex=False)
     .str.strip()
 )

 # Convert required columns
 match["matchDate"] = pd.to_datetime(match["matchDate"], errors="coerce")
 batting["runs"] = pd.to_numeric(batting["runs"], errors="coerce")

 # Remove missing values
 batting = batting.dropna(subset=["batsmanName", "runs", "match_id"])
 match = match.dropna(subset=["matchDate", "winner", "match_id"])

 # Merge batting and match details using match_id
 df = batting.merge(match, on="match_id", how="left")

 print("Total Runs   :", df["runs"].sum())
 print("Total Players:", df["batsmanName"].nunique())
 print("Total Matches:", match["match_id"].nunique())
```

Output:

```
Total Runs   : 11169
Total Players: 198
Total Matches: 45
```

```
 # 1. BAR CHART - Top 10 Run Scorers
 top_batsmen = df.groupby("batsmanName")["runs"].sum().nlargest(10).sort_values()

 plt.figure(figsize=(8, 5))
 top_batsmen.plot(kind="barh", color="steelblue")
 plt.title("Top 10 Run Scorers")
 plt.xlabel("Total Runs")
 plt.ylabel("Batsman Name")
 plt.tight_layout()
 plt.show()
```

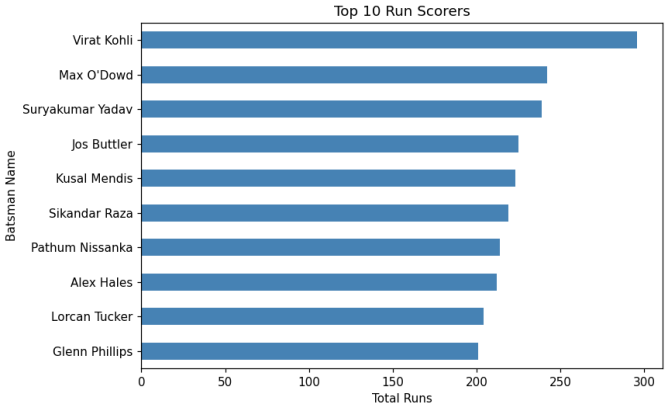


Fig 1: Bar chart - Top 10 Run Scorers

```
 # 2. LINE CHART - Total Runs by Match Date
 daily_runs = df.groupby(df["matchDate"].dt.date)["runs"].sum()

 plt.figure(figsize=(9, 5))
```

```
plt.plot(daily_runs.index, daily_runs.values, marker="o", color="darkorange")
plt.title("Total Runs by Match Date")
plt.xlabel("Match Date")
plt.ylabel("Total Runs")
plt.xticks(rotation=45)
plt.grid(True)
plt.tight_layout()
plt.show()
```

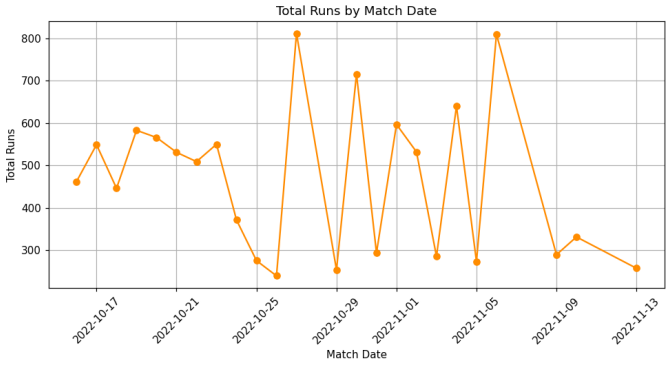


Fig 2: Line chart - Total Runs by Match Date

```
# 3. PIE CHART - Match Wins Distribution
wins = match["winner"].value_counts()

plt.figure(figsize=(8, 6))
plt.pie(wins.values, autopct="%1.1f%%", startangle=90,
        pctdistance=0.8, textprops={"fontsize": 7})
plt.legend(wins.index, loc="center left", bbox_to_anchor=(1, 0.5))
plt.title("Match Wins Distribution")
plt.tight_layout()
plt.show()
```

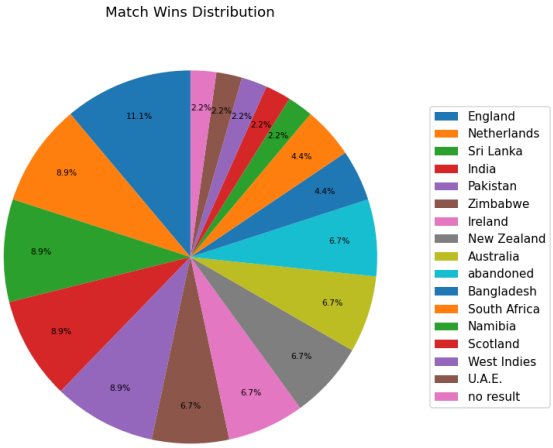


Fig 3: Pie chart - Match Wins Distribution

2. POWER BI

1. Open Power BI Desktop and select Home > Get Data > Text/CSV.
2. Load df_batting.csv and df_match.csv, then open Transform Data and use Replace Values on batsmanName to remove "(c)" and the wrong symbols ("â€").
3. Check the data types of matchDate, runs, batsmanName, and winner.
4. In Model view, create a relationship between df_batting[match_id] and df_match[match_id].
5. Create measures using Table tools > New measure.
   Total Runs = SUM(df_batting[runs])

Total Players = DISTINCTCOUNT(df_batting[batsmanName])
Total Matches = DISTINCTCOUNT(df_match[match_id])

6. Create a bar chart by placing batsmanName on the Y-axis and Sum of runs on the X-axis (Top N filter = 10).
7. Create a line chart using matchDate on the X-axis and Sum of runs on the Y-axis.
8. Create a pie chart using winner in Legend and Count of match_id in Values.
9. Add Card visuals for Total Runs, Total Players, and Total Matches, arrange the visuals, format titles and colors, and save the report as T20_Cricket.pbix.



Fig: T20 Cricket Analytics Dashboard (Power BI)

3. TABLEAU

1. Open Tableau Desktop.
2. Under Connect, select Text File, open df_match.csv, and add df_batting.csv as a second table joined on match_id.
3. Open a new worksheet.
4. Create the bar chart by placing Batsman Name on Rows and Sum of Runs on Columns, then apply a Top 10 filter on Batsman Name by Sum of Runs.
5. For the match results chart, place Match Id on Columns and Winner on Rows, then select Bar in the Marks card.
6. Add chart titles and customize the colors.
7. Select New Dashboard and drag both worksheets onto the dashboard.
8. Save the workbook as T20_Cricket.twbx or .twb.

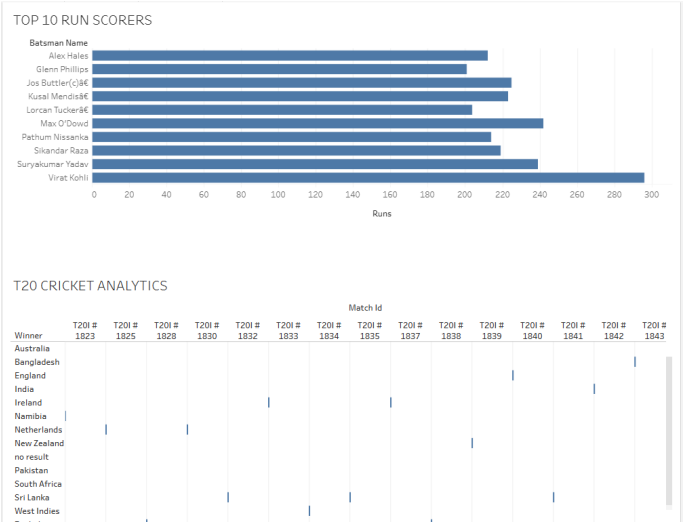


Fig: T20 Cricket Analytics Dashboard (Tableau)

VISUALIZATIONS

1. Bar Chart - Top 10 Run Scorers

Compares the total runs of the leading batsmen to identify the highest run scorer.

2. Line Chart - Total Runs by Match Date

Shows how the total runs scored change across match dates.

3. Pie Chart - Match Wins Distribution

Displays each team's share of the matches won, including matches with no result or abandoned.

OBSERVATION AND INSIGHTS

- Virat Kohli is the highest run scorer with 296 runs, followed by Max O'Dowd (242) and Suryakumar Yadav (239).
- A total of 11,169 runs were scored by 198 unique batsmen across 45 matches. (Power BI shows 200 players because the same player appears under different spellings before the names are cleaned.)
- Total runs by date are highest on 27 Oct 2022 (812 runs) and lowest on 26 Oct 2022 (239 runs).
- England won the most matches (5), followed by India, Netherlands, Pakistan, and Sri Lanka with 4 wins each.
- 4 of the 45 matches (8.9%) ended as abandoned or no result.
- Python, Power BI, and Tableau provide different ways to create and present the same visual analysis.

RESULT

The T20 Cricket dataset was used to create bar charts, line charts, and pie charts using Python, Power BI, and Tableau. The visualizations support identification of top run scorers, analysis of runs by match date, and comparison of match wins across teams using three different data visualization tools.